

Reviewer Pack — Minimal Rank of Quotient Algebras over Boolean Algebras vs D...

Entry e5edd7fc87cb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 14:44:19 UTC

Minimal Rank of Quotient Algebras over Boolean Algebras vs Determinant Circuit Size

Entry ID: e5edd7fc87cb **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 14:44:19 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Quotient Algebras) **Field B** (complexity object): Complexity Theory: Determinant Circuit Complexity

Statement:

[‘For any homogeneous polynomial f over the Boolean algebra B with n generators, there exists a quotient algebra $Q(f)$ constructed such that the minimal rank of $Q(f)$ is $\Theta(n^{\{1.5\}})$, and for all circuits C computing the determinant of a matrix of size $m < n^{1.5}$, the circuit size of C is $\Omega(\rho(Q(f)))$ ’, ‘where $\rho(Q(f))$ denotes the minimal rank of the quotient algebra $Q(f)$.’]

Rationale (proposer’s reasoning):

[‘The use of quotient algebras in algebraic combinatorics allows for a structured analysis of polynomial functions, which may reveal inherent complexities that are not evident from their Boolean representations. The conjecture posits that this structure could be exploited to derive circuit lower bounds for determinant computation.’, ‘This bridge between algebraic combinatorics and complexity theory is relatively unexplored, with only a few papers addressing the connection between quotient algebras and complexity measures.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a0e8fd831b3a8d78

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all $n \leq 40$, the minimal rank of $Q(f)$ is within $\Theta(n^{\{1.5\}})$ and for all $m < n^{1.5}$, the circuit size of C computing the determinant is at least $\Omega(\rho(Q(f)))$. The conjecture is falsified if there exists an $n \leq 40$ such that either the minimal rank of $Q(f)$ deviates from $\Theta(n^{\{1.5\}})$ by more than 10% or for some $m < n^{1.5}$, the circuit size of C is less than $0.5\Omega(\rho(Q(f)))$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'minimal rank quotient algebras boolean algebra' AND 'determinant circuit size' - 'algebraic combinatorics' AND 'circuit complexity determinant' - 'quotient algebra' AND 'Boolean algebra' AND 'complexity theory determinant circuit'

Top relevant hits considered: - [http://arxiv.org/abs/1102.1242v2] A refinement of Stone duality to skew Boolean algebras - [http://arxiv.org/abs/2412.20260v2] Functorial, operadic and modular operadic combinatorics of circuit algebras - [http://arxiv.org/abs/2411.11095v3] Invariant theory and coefficient algebras of Lie algebras - [http://arxiv.org/abs/1711.02729v2] On f- and h- vectors of relative simplicial complexes - [http://arxiv.org/abs/2306.17511v1] Computational Complexity in Algebraic Combinatorics - [http://arxiv.org/abs/1503.04335v2] An algebraic approach to finite projective planes - [http://arxiv.org/abs/math/0501518v2] Deformation of algebras over the Landweber-Novikov algebra - [http://arxiv.org/abs/math/0602046v1] Deformation of dual Leibniz algebra morphisms

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_polynomial(n):
    variables = [f"x{i}" for i in range(1, n+1)]
    coeffs = [random.randint(0, 1) for _ in range(n)]
    polynomial = sum(c * v for c, v in zip(coeffs, variables))
    return polynomial

def generate_matrix(m, n):
    matrix = [[random.choice([0, 1]) for _ in range(n)] for _ in range(m)]
    return matrix

def determinant(matrix):
    if len(matrix) == 2:
        return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
    det = 0
    for i in range(len(matrix)):
        submatrix = [row[:i] + row[i+1:] for row in matrix[1:]]
        sign = (-1) ** (i % 2)
```

```

        det += sign * matrix[0][i] * determinant(submatrix)
    return det

def run_trial(seed):
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        polynomial = generate_polynomial(n)
        Q_f_rank = len(polynomial.split()) # Simplified rank calculation

        for m in range(1, int(math.sqrt(n))**2):
            matrix = generate_matrix(m, n)
            det = determinant(matrix)
            C_size = len(str(det).split())

            results.append({
                "n": n,
                "m": m,
                "polynomial": polynomial,
                "Q_f_rank": Q_f_rank,
                "det": det,
                "C_size": C_size
            })

    total_tests = len(results)
    min_Q_f_rank = min(result["Q_f_rank"] for result in results)
    max_C_size = max(result["C_size"] for result in results)

    conjecture_holds = (min_Q_f_rank >= n_values[0]**1.5 and
                        max_C_size >= 0.5 * min_Q_f_rank)

    return {
        "metric_name": "Conjecture Support",
        "metric_value": (min_Q_f_rank, max_C_size),
        "instances_tested": total_tests,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_Q_f_rank = sum(r["metric_value"][0] for r in results) / len(results)
    mean_C_size = sum(r["metric_value"][1] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):

```

```

    print(f"RESULT: SUPPORTED mean_Q_f_rank={mean_Q_f_rank} mean_C_size={mean_C_size} support_
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
else:
    print("RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_caf51df2.py", line 82, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_caf51df2.py", line 44, in run_trial
    polynomial = generate_polynomial(n)
                  ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_caf51df2.py", line 21, in generate_poly
    polynomial = sum(c * v for c, v in zip(coeffs, variables))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: unsupported operand type(s) for +: 'int' and 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Re-run the test with proper error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14898
2	preregistration	ollama_remote	glm4:latest	0	0	10205
3	novelty	ollama_remote	glm4:latest	0	0	8422
4	novelty	ollama_remote	glm4:latest	0	0	9780
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12987
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12254
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10098

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10051
9	judge	ollama_remote	glm4:latest	0	0	11855

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 100550 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/e5edd7fc87cb.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e5edd7fc87cb.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e5edd7fc87cb.tar.gz (if generated)